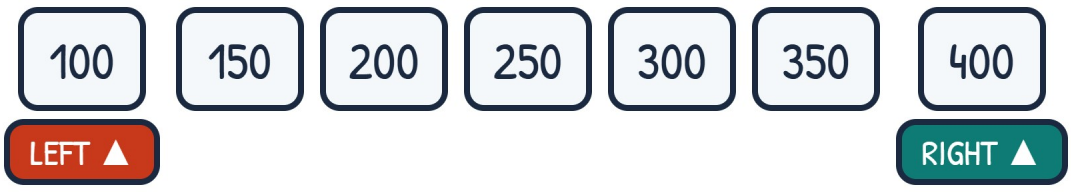
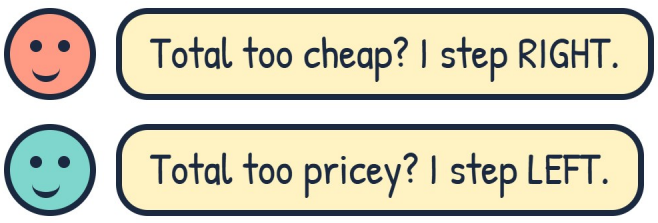
TWO POINTERS

Two fingers instead of two loops: $O(n^2) \rightarrow O(n)$, $O(1)$ space

1. OPPOSITE ENDS

SQUEEZE FROM BOTH SIDES





Real life: a sorted menu, two dishes totalling Rs 500. Never re-check a ruled-out pair.

Spot it: sorted + pair + target sum • palindrome / mirror • max area between two lines

```
while (left < right)
```

2. SAME DIRECTION (FAST & SLOW)

ONE SCOUTS AHEAD, ONE BUILDS BEHIND

A. Reader & Writer (in-place)



Teacher scans the register (fast) and rewrites only new names at the top of the same page (slow).

B. Tortoise & Hare (linked list)



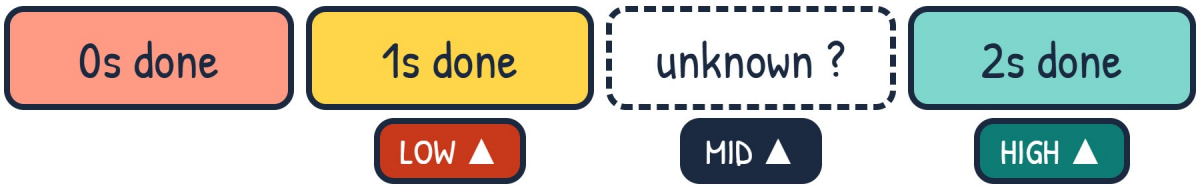
Circular track: hare laps tortoise = **CYCLE**. Straight road: hare finishes, tortoise stands at the **MIDDLE**.

Spot it: in-place, $O(1)$ space, remove / move / keep • linked list: cycle, middle, nth from end

```
while (fast != null && fast.next != null)
```

3. PARTITION / DUTCH FLAG

THREE ZONES, ONE WALK



Real life: sort laundry into 3 baskets in ONE walk past the pile.

Spot it: only 0, 1, 2 (3 colours) · one pass, no library sort · partition around a pivot

Your job: what should happen when mid sees 0, 1, 2? One case is special!

`while (mid <= high)`

4. GROUP COUNTING

COUNT THE CROWD, THEN JUMP THE CROWD

Counting pairs in a sorted array with repeated values:

Ends EQUAL
1 1 1 1 → 6 pairs
n × (n−1) / 2
Handshakes!

Ends DIFFERENT
1 1 | 2 2 → 4 pairs
left copies × right copies
Dance partners!

Then jump both pointers past the groups. 2nd loop: `right >= left`

WHICH ONE?

Ends / pairs in sorted data → **OPPOSITE ENDS**

In-place filter, linked list cycle / middle → **FAST & SLOW**

Only 3 categories, one pass → **DUTCH FLAG**

Count pairs + repeated values → **GROUP COUNTING**

Ask: which element can I decide for certain? Decide it, then shrink.
Trap: unsorted or negatives → not Two Pointers (try a hash map or Kadane).